

Digital Circuits

Exercise 3 - Seven Segment Display

Group/Team No.: G1 T3

Date: December 1st, 2025

Name: Georgii

Name: Luciano

Last Name: Molyboga

Last Name: Carricart

IM No.: 2782258

IM No.: 2782740

Lab Exercise 3	max score	score reached
P1 LED Signal Assignment	4	
T0 Create Project	1	
T1		
T2		
T3		
T4		
T5		
T6 Timing analysis	5	
Sum	40	
Report	15	
Penalty		
Score Exercise 3	55	
Total score DIP (E1-E3)	117	

1 Introduction

1.1 Objectives

The objectives of the experimental session were gaining practical experience in the use of a finite state machine as a structured approach to microcontroller programming, and the encoding/decoding of a 4-bit switch input and a 8-bit LED output.

Furthermore, the inspection of the XC71A35T-1CPG236 FPGA User Manual, in conjunction with the use of counters as a way to control physical time is inspected.

Lastly, an analysis of the performance is enabled through the inspection of a timing diagram and timing summary.

1.2 Task Description

Four different tasks were given for this laboratory experience.

1. Display a four in the first digit of the seven segment display array
2. Display an increasing counter from zero to nine in all digits
3. Achieve the same result as in the previous point, but ensure minimum delay in every cycle repetition
4. Treat the switches as a four-digit binary encoder, and display increasing sequences moving from zero to n, where n is the selected input number. For all inputs higher than nine, show all zero values.

2 Laboratory Preparation

2.1 Basys 3 Board

The FPGA XC71A35T-1CPG236 is a Xilinx, Artix-7 (mid-series) family profile. What defines high/medium/low series are factors such as

- amount of logic elements
- amount of DSP slices
- amount of memory blocks
- amount of built-in features (e.g.: high-speed transceivers)

The board runs with a clock of 100 MHz and includes multiple peripherals enumerated in Figure 1. Among all, the following are selected for further study:

- Pmod port (peripheral module): a handy interface typical in many FPGAs (prototyping/educational). It is standardized and allows to plug-and-play with modules such as OLED screens, I2C interfaces, temperature sensors, and accelerometers.
- Analog signal Pmod port (XADC): built-in ADC that properly maps analog signals. If one were to plug a thermistor to a digital I/O pin, it won't get a smooth range of temperature readings; only realizing a fixed threshold (digital pins may still be used but would require PWM modulation). This is why an ADC pin exists in development boards.

- Programming mode jumper: its configuration defines if the board should flash using JTAG, Flash/SPI, or other sources.
- Shared UART/JTAG USB port: a single USB handling debugging / flashing and UART communication with the PC for an application or command window.
- Power Select Jumper: allows powering through USB or an external source.

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod port(s)	10	Programming mode jumper
3	Analog signal Pmod port (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

Figure 1: Basys 3 board and peripherals

2.2 Seven-segment Display

A mapping between concrete FPGA pins together with the specific control of the seven-segment display needs to be defined through a constraints file. AN0, AN1, AN2, and AN3 act as digit selectors, while CA, CB, CC, CD, CE, CF, CG, DP act as digit definers. Each digit has a common anode that provides positive powering to all of the diodes together, and individual cathodes that decide if the other end of the diode connection should be grounded or set to a high impedance state, controlling ON/OFF states.

2.3 Software

The output port LEDs is mapped to the individual diodes conforming each digit of the seven-segment display, and control the output sequence displayed in a single digit. DIGIT_select, on the other hand, controls which of the four available digits should follow the LEDs output indicated. SW is mapped to the input switches, allowing the user to make physically inputs.

The implemented bit patterns for ZERO - NINE constants are:

- ZERO = "11000000"
- ONE = "11111001"
- TWO = "10100100"
- THREE = "10110000"
- FOUR = "10011001"
- FIVE = "10010010"
- SIX = "10000010"
- SEVEN = "11111000"
- EIGHT = "10000000"
- NINE = "10010000"

3 Part 1 - Displaying a 4

3.1 General Approach

A three-step finite state machine is used to reach a structured solution that will simplify future tasks. Three processes will be defined, a combinatorial "next-state" logic Delta, a sequential state register process, and a combinatorial output logic Lambda.

The Delta process uses combinational logic to calculate the value of the next state signal (counter_ns) based on the current state (counter_cs) and external inputs. It defines the state transition logic, determining how the system moves from one state to the next (incrementing the counter) independent of the clock.

The sequential process implements the system's synchronous memory (flip-flops) to store the current state. It updates the current state signal (counter_cs) with the value of the next state signal (counter_ns) on every rising edge of the clock and handles the asynchronous reset. Within this process, a sensitivity list (clk, rst) is used.

The Lambda process uses combinational logic to decode the current state (counter_cs) and inputs into the final output signals for the LEDs and digit selector. It translates the system's internal binary state into the specific physical bit patterns required to drive the 7-segment display segments and anodes.

3.2 Solution

Delta process

- Counter assignment; since it is not necessary for this trivial exercise, it can be fixed to zero.

Sequential register process

- If the RESET button is pressed, the counter_cs is reset to zero.
- If a clk'event AND clk = '1' (clock transitions to high state), then counter_cs = counter_ns.

Lambda process

- DIGIT_select is set to "0111" in order to light up the first digit only.
- LEDs is assigned the value of FOUR (defined constant)

4 Part 2 - Displaying a Simple Sequence

4.1 Approach

The General Approach structure defined in Part 1 is reused for this exercise and the following ones. The inclusion of a counter is now required to satisfy the requirements of the application. Every digit increment should happen at a frequency between 1 kHz - 60 Hz, which define the limits of human eye detection. This could be achieved in two possible ways, but the use of a 30-bit counter is chosen for this and the remaining task exercises.

As mentioned during in the Preparation section, the board is running at 100 million cycles per second. It is desired to have numbers on the display to stay visible for around a second, needing

the FPGA to wait for roughly 67 million cycles before changing the number. The question that arises is how many bits are needed to count to 67 million?

$$2^{26} = 67,108,864 \text{ (approximately 0.67s)}$$

The use of a 30-bit counter, then, allows to change between 10 different values that change every 0.67 seconds, by only looking at the last 4 bits of the whole counter.

4.2 Solution

Delta process

- The counter is incremented by 1.
- Logic to reset the counter does not need to be implemented because an overflow will result in automatic reset of the variable.
- counter_ns is assigned the value of counter_cs.

Sequential register process

- If the RESET button is pressed, the counter_cs is reset to zero.
- If a clk'event AND clk = '1' (clock transitions to high state), then counter_cs = counter_ns;

Lambda process

- DIGIT_select is fixed to "0000" in order to light up all digits.
- LEDs is assigned constants from ZERO to NINE depending on the value of the counter, using "case" logic.

5 Part 3 - Improvement of a Simple Sequence

5.1 Motivation

The solution provided in Part 2 is effective but not efficient. At the end of every sequence, the counter shows an undesired delay until it restarts back to ZERO. In order to improve this solution, it is needed to include a guard that checks when the last 4 bits of the counter reach "1010", resulting in a counter reset.

5.2 Solution

Delta process

- The counter is incremented by 1.
- If the last 4 bits of the counter equal "1010", then reset the counter.
- counter_ns is assigned the value of counter_cs.

Sequential register process

- If the RESET button is pressed, the counter_cs is reset to zero.
- If a clk'event AND clk = '1' (clock transitions to high state), then counter_cs = counter_ns;

Lambda process

- DIGIT_select is fixed to "0000" in order to light up all digits.
- LEDs is assigned constants from ZERO to NINE depending on the value of the counter, using "case" logic.

6 Part 4 - Switch Decoding and Sequence Display

6.1 Approach

The solution from Part 3 is now adapted to include the decoding of a 4-bit switch input, that will influence the output sequence. In order to achieve this, the sensitivity list of the Delta and Lambda processes need now to include SW, since both need to read its value in order to take decisions.

Changing the SW state before a sequence finishes should reset the counter to zero before the start of the new sequence. This should be addressed carefully, because the FSM should depend on the clock, and not in asynchronous events such as the change of a switch. There are two possible ways of reaching a solution, and considering the time constraints of the laboratory experience, a turnaround was chosen by our group. Theoretically, one could save the previous state of the switch inside the sequential process, and compare it previous state with the current state, allowing a synchronized counter reset. Practically, a condition was added in the delta process that mathematically resets the counter when `counter_cs(29 downto 26) > unsigned(SW(3 downto 0))`.

6.2 Solution

Delta process

- The counter is incremented by 1.
- If the last 4 bits of the counter equal "1010" OR the last 4 bits of the counter are bigger than the decoded input switch number, then reset the counter.
- `counter_ns` is assigned the value of `counter_cs`.

Sequential register process

- If the RESET button is pressed, the `counter_cs` is reset to zero.
- If a `clk'event AND clk = '1'` (clock transitions to high state), then `counter_cs = counter_ns`;

Lambda process

- `DIGIT_select` is fixed to "0000" in order to light up all digits.
- Using a higher layer of "case" logic, the output sequence is defined depending on the value of the switch defined by the user.
- Using a lower layer of "case" logic inside every SW case option, the LEDs sequence is defined depending on the value of the counter.

Considering the complexity of the last exercise, the source code of Task 4 can be found in Appendix A.

7 Timing Analysis

7.1 Motivation

The purpose of this timing analysis is to prove that the digital circuit designed is fast enough to meet the requirements of our application. In other words, all the logic gates used to create a solution involve a critical path that should not exceed our limits. The clock of the Basys 3

Board runs at 100 MHz (10 ns per cycle), at it needs to be proven that this long chain can still complete its calculation comfortably within the 10 ns clock cycle, ensuring the display updates reliably without glitches or timing violations.

7.2 Results

As shown in the Design Timing Summary, the Worst Negative Slack (WNS) is 5.437 ns. This indicates that the critical path has a propagation delay of approximately 4.56 ns, which is well within the 10 ns clock period. There were 0 failing endpoints and the Total Negative Slack (TNS) was 0.000 ns, verifying that the circuit is free of setup and hold violations.

$$\text{Total Time} - \text{Spare Time (Slack)} = \text{Used Time}$$

$$10.000 \text{ ns} - 5.437 \text{ ns} = 4.563 \text{ ns}$$

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.437 ns	Worst Hold Slack (WHS): 0.305 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 30	Total Number of Endpoints: 30	Total Number of Endpoints: 31

All user specified timing constraints are met.

Figure 2: Timing Summary

8 Conclusion

The experience was unique and enriching, enabling students to materialize practical VHDL knowledge covered during theoretical lectures. Several learning outcomes result from the laboratory session.

- The use of finite state machines, an essential block that allows a strict definition of all possible behaviors of a microcontroller application.
- Three-process approach leading to modular, clean, and understandable code.
- Encoding and decoding of signals, specially with the input switch.
- The use of counters as an alternative to measure physical time.
- Synchronicity of responses and the importance of linking processes only to clock signals, and not to asynchronous sources such as the turning of a switch.
- The use of constraints file to map concrete FPGA pins to programmable variables in VHDL.

9 Appendix A - VHDL Code Task 4

```
1  -----
2  -- libraries
3  library ieee;
4  use ieee.std_logic_1164.all;
5  use IEEE.NUMERIC_STD.all;
6
7  -----
8  -- entity
9  entity seven_segment is
10     Port ( clk : in STD_LOGIC;
11           rst : in STD_LOGIC;
12
13           LEDs : out STD_LOGIC_VECTOR(7 downto 0);
14           DIGIT_select : out STD_LOGIC_VECTOR(3 downto 0);
15           SW : in STD_LOGIC_VECTOR(7 downto 0)
16         );
17 end seven_segment;
18
19 -----
20 -- architecture
21 architecture rtl of seven_segment is
22
23
24
25 -----
26 -- signal declaration
27
28 -- Preparation: Update the bit patterns of the constants for the 7-segment
29 -- LEDs. Follows the order dp, g, f, e, d, c, b, a
30 constant ZERO : std_logic_vector(7 downto 0) := "11000000";
31 constant ONE  : std_logic_vector(7 downto 0) := "11111001";
32 constant TWO  : std_logic_vector(7 downto 0) := "10100100";
33 constant THREE : std_logic_vector(7 downto 0) := "10110000";
34 constant FOUR  : std_logic_vector(7 downto 0) := "10011001";
35 constant FIVE  : std_logic_vector(7 downto 0) := "10010010";
36 constant SIX   : std_logic_vector(7 downto 0) := "10000010";
37 constant SEVEN : std_logic_vector(7 downto 0) := "11111000";
38 constant EIGHT : std_logic_vector(7 downto 0) := "10000000";
39 constant NINE  : std_logic_vector(7 downto 0) := "10010000";
40
41 -- Define my counter signal here
42 signal counter_cs, counter_ns : std_logic_vector(29 downto 0);
43
44 begin
45 -----
46 -- sequential process
47 sync : process(clk, rst)
48 begin
49
50 if rst = '1' then
51     counter_cs <= (others => '0');
52 elsif rising_edge(clk) then
53     counter_cs <= counter_ns;
54 end if;
55
56 end process;
57
58 -----
59 -- combinatorial process delta. Initial set / data preparation / preprocessing
60 delta : process(counter_cs, SW)
```



```

61 variable counter_v: std_logic_vector(29 downto 0);
62 begin
63     counter_v := counter_cs;
64
65     if (counter_cs(29 downto 26) = "1010") OR (unsigned(counter_cs(29 downto
66         26)) > unsigned(SW(3 downto 0))) then
67         counter_v := (others => '0');
68     else
69         counter_v := std_logic_vector(unsigned(counter_v) + 1);
70     end if;
71
72     counter_ns <= counter_v;
73 end process;
74
75 -----
76 -- combinatorial process lambda
77 -- Depending on my saved DIGIT_select and LEDs pattern, I want to output the
78 -- correct thing
79 -- Once assigning a value to the output signal, the LEDs will light up
80 lambda : process(counter_cs, SW)
81 begin
82     DIGIT_select <= "0000";
83
84     case SW (3 downto 0) is
85         when "0000" => LEDs <= ZERO;
86         when "0001" =>
87             case counter_cs (29 downto 26) is
88                 when "0000" => LEDs <= ZERO;
89                 when "0001" => LEDs <= ONE;
90                 when others => LEDs <= ZERO;
91             end case;
92         when "0010" =>
93             case counter_cs (29 downto 26) is
94                 when "0000" => LEDs <= ZERO;
95                 when "0001" => LEDs <= ONE;
96                 when "0010" => LEDs <= TWO;
97                 when others => LEDs <= ZERO;
98             end case;
99         when "0011" =>
100             case counter_cs (29 downto 26) is
101                 when "0000" => LEDs <= ZERO;
102                 when "0001" => LEDs <= ONE;
103                 when "0010" => LEDs <= TWO;
104                 when "0011" => LEDs <= THREE;
105                 when others => LEDs <= ZERO;
106             end case;
107         when "0100" =>
108             case counter_cs (29 downto 26) is
109                 when "0000" => LEDs <= ZERO;
110                 when "0001" => LEDs <= ONE;
111                 when "0010" => LEDs <= TWO;
112                 when "0011" => LEDs <= THREE;
113                 when "0100" => LEDs <= FOUR;
114                 when others => LEDs <= ZERO;
115             end case;
116         when "0101" =>
117             case counter_cs (29 downto 26) is
118                 when "0000" => LEDs <= ZERO;
119                 when "0001" => LEDs <= ONE;
120                 when "0010" => LEDs <= TWO;
121                 when "0011" => LEDs <= THREE;
122                 when "0100" => LEDs <= FOUR;

```

```

122         when "0101" => LEDs <= FIVE;
123         when others => LEDs <= ZERO;
124     end case;
125 when "0110" =>
126     case counter_cs (29 downto 26) is
127     when "0000" => LEDs <= ZERO;
128     when "0001" => LEDs <= ONE;
129     when "0010" => LEDs <= TWO;
130     when "0011" => LEDs <= THREE;
131     when "0100" => LEDs <= FOUR;
132     when "0101" => LEDs <= FIVE;
133     when "0110" => LEDs <= SIX;
134     when others => LEDs <= ZERO;
135     end case;
136 when "0111" =>
137     case counter_cs (29 downto 26) is
138     when "0000" => LEDs <= ZERO;
139     when "0001" => LEDs <= ONE;
140     when "0010" => LEDs <= TWO;
141     when "0011" => LEDs <= THREE;
142     when "0100" => LEDs <= FOUR;
143     when "0101" => LEDs <= FIVE;
144     when "0110" => LEDs <= SIX;
145     when "0111" => LEDs <= SEVEN;
146     when others => LEDs <= ZERO;
147     end case;
148 when "1000" =>
149     case counter_cs (29 downto 26) is
150     when "0000" => LEDs <= ZERO;
151     when "0001" => LEDs <= ONE;
152     when "0010" => LEDs <= TWO;
153     when "0011" => LEDs <= THREE;
154     when "0100" => LEDs <= FOUR;
155     when "0101" => LEDs <= FIVE;
156     when "0110" => LEDs <= SIX;
157     when "0111" => LEDs <= SEVEN;
158     when "1000" => LEDs <= EIGHT;
159     when others => LEDs <= ZERO;
160     end case;
161 when "1001" =>
162     case counter_cs (29 downto 26) is
163     when "0000" => LEDs <= ZERO;
164     when "0001" => LEDs <= ONE;
165     when "0010" => LEDs <= TWO;
166     when "0011" => LEDs <= THREE;
167     when "0100" => LEDs <= FOUR;
168     when "0101" => LEDs <= FIVE;
169     when "0110" => LEDs <= SIX;
170     when "0111" => LEDs <= SEVEN;
171     when "1000" => LEDs <= EIGHT;
172     when "1001" => LEDs <= NINE;
173     when others => LEDs <= ZERO;
174     end case;
175 when others => LEDs <= ZERO;
176 end case;
177
178 end process;

```